

HashSet → Uniqueness / Duplicates

HashMap → Frequency / Mapping / Complement

int[26] → Small fixed alphabet (a–z)

Prefix/Suffix → Left × Right contribution

1. INTERVIEW FLOW (ALWAYS FOLLOW)

1 Clarify — Constraints, edge cases, input range, duplicates?

2 Brute Force — Write simple solution first

3 Better — Can we optimize time?

4 Optimal — Best approach (Time/Space)

5 Complexity — Analyze Time & Space

6 Code — Clean & bug-free

7 Explain — Walk through / dry run

3. PROBLEMS AT A GLANCE (9 CORE PROBLEMS)

LC#	Problem	Pattern	Key Idea	Time	Space
217	Contains Duplicate	HashSet	If !set.add(x) → duplicate	O(n)	O(n)
242	Valid Anagram	int[26] / HashMap	Count freq of both strings	O(n)	O(1)
1	Two Sum	HashMap	Store num→index; check target-num	O(n)	O(n)
49	Group Anagrams	HashMap	Sort string or count freq as key	O(n·k log k)	O(n·k)
347	Top K Frequent Elements	HashMap + Heap	Count freq; Min Heap of size k	O(n log k)	O(n)
271	Encode & Decode Strings	StringBuilder	Length#string format; parse length first	O(n)	O(n)
238	Product Except Self	Prefix / Suffix	Left[i] × Right[i]; no division	O(n)	O(1)*
128	Longest Consecutive Seq.	HashSet	Start count only if num-1 not in set	O(n)	O(n)
36	Valid Sudoku	HashSet	3 sets: rows, cols, 3×3 boxes	O(1)	O(1)
—	Character Count (bonus)	int[26]	index = char - 'a' (0 to 25)	O(n)	O(1)

2. PATTERN RECOGNITION (QUICK LOOKUP)

Need uniqueness / duplicates?	HashSet
Need frequency count?	HashMap / int[26]
Small fixed alphabet (a–z)?	int[26]
Need grouping?	HashMap<Key, List>
Need Top K frequent?	HashMap + Heap
Need Left & Right products?	Prefix / Suffix
Need sequence (consecutive)?	HashSet
Need matrix validation?	HashSet per row/col/box
Need encoding / decoding?	Length + Delimiter (#)

4. COMPLEXITY QUICK REFERENCE

HashSet add/contains/remove	O(1) avg
HashMap get/put/containsKey	O(1) avg
Array access	O(1)
For loop (n times)	O(n)
Two separate loops	O(2n) = O(n)
Nested loops (n×n)	O(n²)
Sorting (Arrays.sort)	O(n log n)
Heap offer/poll	O(log n)
Bucket Sort (by frequency)	O(n)

5. BIG-O RULES (DON'T FORGET)

Sequential loops → ADD

for i in n O(n)
for j in n O(n)
= O(2n) = O(n)

Nested loops → MULTIPLY

for i in n O(n)
for j in n × O(n)
= O(n²)

Always drop constants. Focus on growth rate.

6. WHEN TO USE WHAT?

Check existence / uniqueness	HashSet
Count frequency / key-value	HashMap
Only lowercase letters a–z	int[26]
Top K / Priority based	Heap (PQ)
Left/Right contribution	Prefix/Suffix
Consecutive sequence	HashSet (seq.)
Row / Col / Box validation	HashSet (matrix)

7. MEMORY MANTRAS (QUICK RECALL)

Need Unique?	Need Count?	Need Left & Right?	Need Top K?
HashSet	HashMap	Prefix / Suffix	Heap
Need Consecutive?	Need Grouping?	Need Encoding?	
HashSet	HashMap<Key,List>	length#string	

8. COMMON PITFALLS (AVOID)

- ✗ Using sort() when HashSet solves in O(n)
- ✗ Forgetting to check num-1 before counting sequence (LC 128)
- ✗ Using split() in Encode/Decode — causes errors with '#'
- ✗ Not handling empty input or negative numbers
- ✗ Wrong comparator in PriorityQueue
- ✗ Not clearing data structure in multiple test cases
- ✗ Not analyzing time & space complexity
- ✗ Over-complicating when a simpler pattern fits

9. TIPS & TRICKS

- ✓ Think pattern first, code later
- ✓ Can HashSet replace sorting? Always ask!
- ✓ Avoid repeated work (like in Longest Consecutive)
- ✓ Use freq array for small fixed ranges (0–25)
- ✓ For Top K → use Min Heap of size K
- ✓ Edge cases decide AC/TLE. Handle early
- ✓ Clarity > Cleverness in interviews
- ✓ Always analyze both Time and Space

ARRAYS & HASHING – 8 CORE PATTERNS

When to Use · Key Idea · AHA Moment · Java Template · Complexity & Examples

HashSet → Uniqueness HashMap → Frequency / Mapping int[26] → Fixed alphabet Prefix/Suffix → Left × Right ★ Master these 8 — solves 80% of A&H problems

PATTERN 1 DUPLICATE DETECTION (LC 217 Contains Duplicate, LC 219, LC 220)				
WHEN TO USE? - Check if any element appears more than once - Find seen/visited elements - Detect repeats in array	DS TO USE HashSet Insert into set; if already present → duplicate found	IDEA / AHA <i>Add to set while iterating. If add() returns false → duplicate found!</i>	JAVA TEMPLATE <pre>Set<Integer> set = new HashSet<>(); for (int x : nums) { if (!set.add(x)) // add() returns false if already exists return true; // duplicate found } return false;</pre>	COMPLEXITY Time: O(n) Space: O(n) LC 217 Contains Duplicate LC 219 Contains Duplicate II
PATTERN 2 FREQUENCY COUNTING (LC 242 Valid Anagram, LC 347 Top K, LC 49 Group Anagrams)				
WHEN TO USE? - Count occurrences of elements - Map each element to its frequency - Anagrams, majority element, Top K	DS TO USE HashMap or int[26] (int[26] for lowercase a-z only)	IDEA / AHA <i>Map each element → its frequency. Helps in anagrams, top K freq, majority element etc.</i>	JAVA TEMPLATE A) HashMap (any values) <pre>Map<Integer,Integer> map = new HashMap<>(); for (int x : nums) map.put(x, map.getOrDefault(x, 0) + 1);</pre> B) int[26] (chars 'a'-'z') <pre>int[] freq = new int[26]; for (char c : s.toCharArray()) freq[c - 'a']++;</pre>	COMPLEXITY Time: O(n) Space: O(n) / O(1) LC 242 Valid Anagram LC 347 Top K Frequent LC 49 Group Anagrams
PATTERN 3 COMPLEMENT / PAIR SEARCH (LC 1 Two Sum, LC 167, LC 560 Subarray Sum = K)				
WHEN TO USE? - Find pair with given sum / target - For each x, check if (target - x) exists - Store seen elements with their index	DS TO USE HashMap (value → index) For each num, check if target - num exists	IDEA / AHA <i>For each x, check if (target - x) exists in map. Store seen elements with their index.</i>	JAVA TEMPLATE (TWO SUM) <pre>Map<Integer,Integer> map = new HashMap<>(); for (int i = 0; i < nums.length; i++) { int comp = target - nums[i]; if (map.containsKey(comp)) return new int[]{map.get(comp), i}; map.put(nums[i], i); }</pre>	COMPLEXITY Time: O(n) Space: O(n) LC 1 Two Sum II LC 167 Two Sum II LC 560 Subarray Sum=K
PATTERN 4 GROUPING / BUCKETING (LC 49 Group Anagrams, LC 438 Find All Anagrams)				
WHEN TO USE? - Group elements by key (anagram, index diff, etc.) - Map key → list of elements - Very useful in anagram problems	DS TO USE HashMap<Key, List> Key = sorted string or 26-char freq string	IDEA / AHA <i>Map key → list of elements. Anagrams share the same sorted key or freq key.</i>	JAVA TEMPLATE (GROUP ANAGRAMS) <pre>Map<String,List<String>> map = new HashMap<>(); for (String s : strs) { char[] arr = s.toCharArray(); Arrays.sort(arr); String key = new String(arr); map.computeIfAbsent(key, k -> new ArrayList<>()).add(s); } return new ArrayList<>(map.values());</pre>	COMPLEXITY Time: O(n·k log k) Space: O(n·k) (k = avg string length) LC 49 Group Anagrams LC 438 All Anagrams
PATTERN 5 PREFIX / SUFFIX TECHNIQUE (LC 238 Product Except Self, LC 724 Find Pivot Index)				
WHEN TO USE? - Product except self, prefix sums, suffix sums - Precompute left and right contributions - Combine to get result in O(n) without extra structures	DS TO USE Prefix & Suffix Arrays ans[i] = left[i] × right[i] No division needed!	IDEA / AHA <i>Brute Force: Left[] → Right[] → Left × Right. Result[i] = prefix product × suffix product. No division. O(n) time, O(1) space.</i>	JAVA TEMPLATE (PRODUCT EXCEPT SELF) <pre>int n = nums.length; int[] left = new int[n], right = new int[n], ans = new int[n]; left[0] = 1; for (int i = 1; i < n; i++) left[i] = left[i-1] * nums[i-1]; right[n-1] = 1; for (int i = n-2; i >= 0; i--) right[i] = right[i+1] * nums[i+1]; for (int i = 0; i < n; i++) ans[i] = left[i] * right[i];</pre>	COMPLEXITY Time: O(n) Space: O(1)* *excl. output LC 238 Product Except Self LC 724 Pivot Index
PATTERN 6 ENCODING / SERIALIZATION (LC 271 Encode & Decode Strings)				
WHEN TO USE?	PATTERN	IDEA / AHA	JAVA TEMPLATE	COMPLEXITY

- Encode list of strings into single string - Decode back to original list - Avoid ambiguity (e.g., "ab#c" vs "a#bc")	length#string Prefix each string with its length and '#' delimiter. <i>Never use split(). Parse length first.</i>	<i>"#" alone causes bugs. length#string avoids all ambiguity. Always parse length first!</i>	Encode <pre>StringBuilder sb = new StringBuilder(); for (String s : strs) sb.append(s.length()).append('#').append(s); return sb.toString();</pre> Decode <pre>List<String> res = new ArrayList<>(); int i = 0; while (i < s.length()) { int j = i; while (s.charAt(j) != '#') j++; int len = Integer.parseInt(s.substring(i, j)); res.add(s.substring(j + 1, j + 1 + len)); i = j + 1 + len; }</pre>	Time: O(n) Space: O(n) LC 271 Encode & Decode Strings
---	--	--	--	--

PATTERN 7 LONGEST CONSECUTIVE SEQUENCE (LC 128 — Hard)				
WHEN TO USE? - Find length of longest consecutive sequence - Only start counting from the first element of a sequence (num-1 not present)	DS TO USE HashSet Store all numbers in set. Start counting only if num-1 not in set.	IDEA / AHA <i>Start count only if num-1 is absent. This avoids repeated traversals. Each number visited at most twice → O(n).</i>	JAVA TEMPLATE <pre>Set<Integer> set = new HashSet<>(nums); // add all nums int longest = 0; for (int num : set) { if (!set.contains(num - 1)) { // start of sequence int cur = num, count = 1; while (set.contains(cur + 1)) { cur++; count++; } longest = Math.max(longest, count); } } return longest;</pre>	COMPLEXITY Time: O(n) Space: O(n) LC 128 Longest Consecutive Seq.

PATTERN 8 MATRIX VALIDATION (SUDOKU) (LC 36 Valid Sudoku)				
WHEN TO USE? - Validate rows, columns, and 3×3 sub-boxes - No duplicates in rows, columns and boxes	DS TO USE HashSet per row/col/box 3×3 Box: (row/3)*3 + col/3	IDEA / AHA <i>3 steps: Check rows, columns, 3×3 boxes. Use set.clear() after each check. Box traversal: jump by 3.</i>	JAVA TEMPLATE (3×3 BOX TRAVERSAL) <pre>// 3x3 box traversal for (int r = 0; r < 9; r += 3) { for (int c = 0; c < 9; c += 3) { Set<Character> seen = new HashSet<>(); for (int i = r; i < r + 3; i++) { for (int j = c; j < c + 3; j++) { char ch = board[i][j]; if (ch == '.') continue; if (!seen.add(ch)) return false; } } } }</pre>	COMPLEXITY Time: O(1) Space: O(1) (fixed 9×9 board) LC 36 Valid Sudoku

★ **GOLDEN RULE:** Understand the Pattern → Choose the Right Data Structure → Code Clean → Analyze Complexity → Optimize if Needed

Next: Java APIs & Templates →

ARRAYS & HASHING – JAVA APIS & TEMPLATES

PAGE
3

Java Collections · Conversions · Key Templates · Common Mistakes · AHA Moments

🔑 **computeIfAbsent** → Very Important!

🔑 **getOrDefault** → Always use instead of manual null check

🔑 **merge()** → `map.merge(key, 1, Integer::sum)`

1. HASHSET<E> — UNIQUE ELEMENTS

```
Set<Character> set = new HashSet<>();
set.add('a'); // true if added
set.contains('a'); // O(1) avg
set.remove('a');
set.size();
set.isEmpty();
set.clear();
// Time: add/contains/remove → O(1) avg
```

2. HASHMAP<K,V> — KEY → VALUE

```
Map<String,Integer> map = new HashMap<>();
map.put("a", 1); // insert/update
map.get("a"); // returns value or null
map.getOrDefault("a", 0); // default if not present
map.containsKey("a");
map.computeIfAbsent("a", k -> new ArrayList<>());
// If key absent → creates value; else adds to existing
map.merge(key, 1, Integer::sum); // increment
map.keySet(); map.values(); map.entrySet();
map.size(); map.isEmpty(); map.clear();
// Time: put/get/containsKey → O(1) avg
```

3. LINKEDHASHMAP — INSERTION ORDER

```
Map<Integer,String> lhm = new LinkedHashMap<>();
lhm.put(1, "a"); lhm.put(2, "b");
// Keys iterated in insertion order: 1, 2
// Time Complexity: O(Log n)
```

4. TREEMAP<K,V> — SORTED BY KEY

```
Map<Integer,String> tm = new TreeMap<>();
tm.put(3, "c"); tm.put(1, "a"); tm.put(2, "b");
// Keys iterated in sorted order: 1, 2, 3
// Time Complexity: O(Log n)
```

5. COMMON JAVA CONVERSIONS (QUICK RECALL)

String ↔ char[]
`String → char[]` `s.toCharArray()`
`char[] → String` `new String(arr)`

String ↔ int
`String → int` `Integer.parseInt(s)`
`int → String` `String.valueOf(n)` / `Integer.toString(n)`

char ↔ int
`char → int` `c - '0'` (digit) / `c - 'a'` (index)

int → char `(char)(n + '0')` or `(char)(n + 'a')`

List ↔ Array
`List → Array` `list.toArray(new Integer[0])`
`Array → List` `Arrays.asList(arr)`

Array → List (mut) `new ArrayList<> (Arrays.asList(arr))`

Set ↔ List
`List → Set` `new HashSet<>(list)`
`Set → List` `new ArrayList<>(set)`

6. PRIORITYQUEUE (MIN HEAP BY DEFAULT)

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.add(5); pq.add(1); pq.add(3);
pq.peek(); // 1 (min)
pq.poll(); // 1 (removes min)
pq.size(); pq.isEmpty();
// For Max Heap:
PriorityQueue<Integer> pqMax = new PriorityQueue<>(
    Collections.reverseOrder());
// Time: add/poll/peek → O(Log n)
```

7. LAMBDA EXPRESSIONS (QUICK)

computeIfAbsent

```
map.computeIfAbsent(key, k -> new ArrayList<>());
```

Comparator (Min Heap by frequency)

```
(a, b) -> freq.get(a) - freq.get(b)
```

Comparator (Max Heap)

```
(a, b) -> a[1] - b[1]; // min heap by 2nd element
```

Sort (custom)

```
arr.sort((a, b) -> b - a); // reverse sort
Arrays.sort(arr, Comparator.reverseOrder());
```

Sort Map by Value

```
map.entrySet().stream()
    .sorted((a,b) -> b.getValue() - a.getValue())
    .collect(Collectors.toList());
```

8. TOP K FREQUENT TEMPLATE (BUCKET SORT)

```
Map<Integer,Integer> freq = new HashMap<>();
for (int n : nums) freq.put(n, freq.getOrDefault(n,0)+1);

List<Integer>[] bucket = new List[nums.length + 1];
for (int num : freq.keySet()) {
    int f = freq.get(num);
    if (bucket[f] == null) bucket[f] = new ArrayList<>();
    bucket[f].add(num);
}

List<Integer> res = new ArrayList<>();
for (int f = bucket.length-1; f >= 0 && res.size() < k; f--)
    if (bucket[f] != null) res.addAll(bucket[f]);
return res.subList(0, k);
```

⚡ Bucket Sort is O(n) time! (vs O(n log k) for heap approach)

9. COMMON MISTAKES (AVOID!)

- ✗ Using sort when HashSet works in O(n)
- ✗ Forgetting to check num-1 before counting sequence (LC 128)
- ✗ Using split() in Encode/Decode — TLE & bugs with '#'
- ✗ Not handling negative numbers or edge cases
- ✗ Wrong comparator: a-b overflow risk for large ints
- ✗ Not clearing data structure between test cases
- ✗ Assuming HashMap is ordered (use LinkedHashMap)

10. AHA MOMENTS (SAVE TO MEMORY)

- ★ **Group Anagrams:** `key.append(freq).append('#')` — '#' avoids ambiguity
- ★ **Top K Frequent:** HashMap → Min Heap of size K
- ★ **Product Except Self:** Brute Force: Left[] → Right[] → Left×Right; Result[i]=rightProduct
- ★ **Longest Consecutive:** Sorting? NO → !contains(num-1) → while(contains++)
- ★ **Encode & Decode:** `length#string` — never use split(). Parse length first.
- ★ **Can I use HashSet to replace sorting?** — Always ask!
- ★ **Hashing gives O(1) avg lookup** but O(n) worst case (rare with good hash)

11. CHECKLIST – WHEN SOLVING

- ✓ Understand constraints (n, values, range)
- ✓ Think: Can I use HashSet for O(1) lookups?
- ✓ Need counts? → HashMap or int[26]
- ✓ Need order? → LinkedHashMap / TreeMap
- ✓ Top K / Min/Max? → PriorityQueue (Heap)
- ✓ Need grouping? → HashMap<Key, List>
- ✓ Need encoding? → length#string format
- ✓ Can I do prefix/suffix to avoid re-computation?
- ✓ Always analyze both Time and Space complexity

12. INTERVIEW COMMUNICATION TIPS

- ✓ Think out loud. Explain each step.
- ✓ Start with brute force, then optimize.
- ✓ Give examples and dry run.
- ✓ State assumptions clearly.
- ✓ Write clean, readable code.
- ✓ Summarize time & space at the end.
- ✓ Handle edge cases early.
- ✓ Patterns > Problems. Master the pattern!

Golden Line: "Brute Force → Better → Optimal → Complexity → Code → Explain"

13. SPACE RULE OF THUMB

- HashSet / HashMap → O(n)
- Frequency Array (26) → O(1)
- Prefix / Suffix → O(n) or O(1) (extra excluding output)
- int[26] with c - 'a' (extra space = O(1))

14. FAANG COMPLEXITY SUMMARY TABLE

Approach	Time	Space
Brute Force (Nested Loop)	O(n ²)	O(1)
Hashing (Set/Map)	O(n)	O(n)

15. ARRAYS UTILITY (MOST USED)

```
Arrays.sort(arr);
Arrays.sort(arr, Comparator.reverseOrder());
Arrays.fill(arr, 0);
Arrays.copyOf(arr, newLength);
Arrays.equals(arr1, arr2);
Arrays.stream(arr).boxed()
    .collect(Collectors.toList());
```

16. QUEUE / DEQUE / STACK

```
// Queue
Queue<Integer> q = new LinkedList<>();
q.offer(1); q.poll(); q.peek();
// Deque (double-ended)
Deque<Integer> dq = new ArrayDeque<>();
dq.offerFirst(1); dq.offerLast(2);
dq.pollFirst(); dq.pollLast();
// Stack (use Deque – preferred)
Deque<Integer> st = new ArrayDeque<>();
st.push(1); st.pop(); st.peek();
```

Sorting	O(n log n)	O(1)/O(n)
Heap (Top K)	O(n log k)	O(k)
Bucket Sort	O(n)	O(n)
Design		

★ MOST USED: HashSet · HashMap + computeIfAbsent · getOrDefault / merge · PriorityQueue · Arrays.sort · StringBuilder

PRACTICE. PATTERNS. REPEAT. CRACK FAANG! 🚀